

TCP – Notes & Additional Slides

Recap: *TCP within the overall context of
the TCP/IP internetworking model*

שכבת ה-transport, TCP ו-UDP, מספקת שירות לשכבת האפליקציה [= שכבה מספר 5]
או, ליתר דיוק: לפרוטוקולים של שכבת האפליקציה

- 2 סוגים של שירותים:

- שירות שהוא גם connection oriented וגם העברת נתונים אמינה [=RDT]. TCP.
- שירות שהוא גם connectionless וגם העברת נתונים לא אמינה. UDP.

- השירות ששכבה ה-Transport מספקת לפרוטוקולי שכבה 5 'מונגש' דרך/ע"י ממשק הסוקטים [= sockets API]

- סוגים שונים של sockets APIs תוכננו ומומשו. ובכלל זה:

- BSD sockets

- Winsock

בקורס אנחנו מתמקדים ב-BSD sockets API.

פרוטוקולי שכבה 5 אינם 'מתקיימים' ב'פני עצמם', אלא הם חלק מאפליקציות רשתיות

אפליקציה רשתית = לפחות 2 תכונות שונות שרצות במארחים שונים ששייכים לרשת הווירטואלית התכניות השונות מחליפות מידע לפי אוסף כללים/חוקים שנקרא פרוטוקול תקשורת. פרוטוקול שכבה 5. קיימים מודלים שונים של אפליקציות רשתיות מודל לקוח-שרת מודל עמית-לעמית מודל משולב אנחנו בקורס מתמקדים באפליקציות לקוח-שרת בלבד.

באפליקציות אלו תכנית אחת נקראת תכנית הלקוח והשנייה נקראת תכנית השרת. הן, כאמור, מחליפות הודעות ביניהן לפי הכללים של פרוטוקול שכבה 5. שגם הם, מטבע הדברים, מייצג מודל לקוח-שרת: לפרוטוקול יש צד לקוח ויש צד שרת

יחידות המידע המוחלפות בין צד הלקוח של הפרוטוקול ובין צד השרת שלו נקראות הודעות.

- ואם 'מחברים' את כל שנאמר כד כה:
- שכבת ה-Transport מספקת שירות של end-to-end לפרוטוקולי שכבה 5.
- ז"א, שכבת ה-transport לא מספקת שירות של end-to-end ישירות לאפליקציות הרשתיות. אלא,
'בעקיפין': 'דרך' פרוטוקולי שכבה 5 שמממשים את העברת ההודעות בין 'חלקי האפליקציה' השונים [=]
בין תכנית הלקוח ותכנת השרת].

- והשירות 'מונגש' לפרוטוקולי שכבה 5 על ידי ה- sockets API
- ולפיכך, יש 2 סוגי סוקטים: סוקטים של TCP וסוקטים של UDP
- ולכן, יש פונקציה [socket() =] שמיועדת ליצירת/הקצאת סוקט. שמאפשרת יצירת סוקט TCP או
UDP.

- ויש פעולות [= פונקציות] שמיועדות לסוקטים של TCP.
- לדוגמא: connect(), bind(), send(), receive(),
- ויש פעולות [= פונקציות] שמיועדות לסוקטים של UDP.
- ... sendto(), recvfrom(), ...

ומכלול הפונקציות השונות מהווה את ממשק ה- sockets API

הניסוח הוא:

השירות ששכבה ה- Transport מספקת שירותים לפרוטוקולי שכבה 5.
'ודרכם' {דרך פרוטוקולי שכבה 5} לאפליקציות הרשתיות.
ודרכן {דרך האפליקציות הרשתיות} למשתמשי קצה

ש: איך שכבה 4 מספקת שירות e-2-e לפרוטוקולי שכבה 5?
ת: 'מונגש' דרך/ע"י ממשק הסוקטים [= sockets API]

הקונטקסט של הדיון שלנו ב-TCP הוא כדלקמן:

- 1. TCP מספק שירות e-2-e לפרוטוקולי שכבה 5. ו'דרכם'
 - לאפליקציות רשתיות.
 - שמספקות שירותי 'העברת מידע' למשתמשים אנושיים.
- 2, השירות של TCP יתואר בהקשר של ממשק ה-BSD sockets.
- 3. השירות של TCP יתואר בהקשר של אפליקציות רשתיות מסוג לקוח-שרת. אפליקציה רשתית כזו 'משתמשת' בפרוטוקול שכבה 5 להחלפת מידע בין 'חלקי האפליקציה'. היינו, בין תכנית הלקוח לתכנית השרת. ולכן, כפי שראינו בהרצאות קודמות, לפרוטוקול שכבה 5 כזה יש 2 מרכיבים:
 - צד הלקוח של הפרוטוקול. L5-Prot-client
 - צד השרת של הפרוטוקול. L5-Prot-server

צד הלקוח של הפרוטוקול L5-Prot-client ממומש כחלק מהמכלול הכולל של כלל הקוד של תכנית הלקוח

צד השרת של הפרוטוקול. L5-Prot-server ממומש כחלק מהמכלול הכולל של כלל תכנית השרת

לדוגמא: פרוטוקול ה-HTTP
מסמך האפיון, של IETF|כולל פרק שמתאר את HTTP-client
וכמו כן כולל פרק {אחר !} שמתאר את HTTP-server

The Sockets API

האפיון של HTTP-server כפי שמתואר במסמך האפיון, 'מתורגם'/ממומש לקוד. למשל, בשפת C.
כחלק מכלל הקוד של תכנית WSP [= Web Server program]
והמימוש, של HTTP-server כחלק מתכנית ה-WSP: כולל קריאות לפונקציות: `socket()`, `bind()`,
`listen()`, `accept()`, `send()`, `receive()` וכו'

וקבוצת הפונקציות `socket()`, `connect()`, `send()`, `receive()`, `bind()`, `listen()`, `accept()`, ...
ממה שקרוי BSD Sockets API
ופונקציות אלו ממומשות ע"י TCP client ו-TCP server

הערה: ה-BSD Sockets API כולל גם פונקציות ש'מיועדות לשימוש' של UDP client ו-UDP server
לדוגמא: `sendto()`, `receivefrom()`

כדי להסביר/לחדד מה הכוונה שממשק הסוקטים 'מיועד' עבור מתכנתים לצורך פיתוח/כתיבה של תכניות רשתיות, נשתמש בדוגמא יותר מוכרת לסטודנטים בקורס: פעולות על קבצים.

למתכנת בשפת C הוגדר ממשק (API) שמאפשר לו עבודה עם קבצים ועם התקני ק"פ
מה פירוש 'מאפשר'? הוא יכול בתוכניות שהוא כותב 'לשלב' קריאות לפונקציות מהממשק.
למשל: ל- `scan()`, `fscan()`, `fread()`, ...

ש: מה זה 'ממשק לעבודה עם קבצים'?
ת: אוסף של פונקציות שהמתכנת יכול לקרוא להן כשהוא צריך 'לשלב' בתוכניות שהוא כותב עבודה עם
קבצים

זה המושג של ממשק: אוסף של פונקציות + 'דברים נוספים' {כמו למשל הגדרה של קבועים כמו `stdin`,
`stdout` וכו'. או קבצי הגדרה כמו `stdio.h`}

בנוסף קיים כמובן התיאור מה כל פונקציה בממשק עושה.

ולכן, מתכנת שכותב תכנית שמבצעת פעולות על קבצים, 'משלב' בקוד שלו קריאות לפונקציות ששייכות לממשק הזה.

ש: איזה פונקציות הוא משלב בתוכנית שלו? בהתאם למה שהוא רוצה להשיג ובהתאם למה שכל פונקציה בממשק מבצעת.

ולכן, מתכנת שכותב תכנית שמבצעת החלפת מידע עם תכנית אחרת, 'משלב' בקוד שלו קריאות לפונקציות ששייכות לממשק ה-BSD sockets. ש: איזה פונקציות? בהתאם למה שהוא רוצה להשיג ובהתאם למה שכל פונקציה בממשק מבצעת.

על הקשר שבין ממשק הסוקטים $[Sockets API =]$
לבין הממשק שמספקת מערכת ההפעלה לתכניות $[OS API =]$

מערכת ההפעלה מספקת לתכניות ממשק {OS API =}
שזה: אוסף פונקציות. מאות ואלפי פונקציות.
לכל אחת: תחביר + יש הגדרה מה היא עושה

את הפונקציות האלו אפשר לחלק חלוקת משנה:
פונק' שמספקות פעולות על קבצים
פונק' שמספקות העברת מידע בין תכניות {socket API =}
פונק' שמספקות פעולות על תכנית: הרצת תכנית, סיום ריצת תכנית, וכו'.
פונק' שמספקות פעולות 'גרפיות'
...

על הקשר שבין פרוטוקולי שכבה 5 לבין ממשק הסוקטים

אפליקציה רשתית:

client program <-----> server program

לשם כך: חייבים לתכנן פרוטוקול שכבה 5, או 'לאמץ' פרוטוקול קיים.
חייבים ל'תכנת' {'לתרגם' לקוד} את האפיון של הפרוטוקול.

איך? מהם כלי העזר שעומדים לרשות המתכנת לצורך 'תרגום לקוד'? ממשק הסוקטים!!

L5-prot-client <-----> L5-prot-server

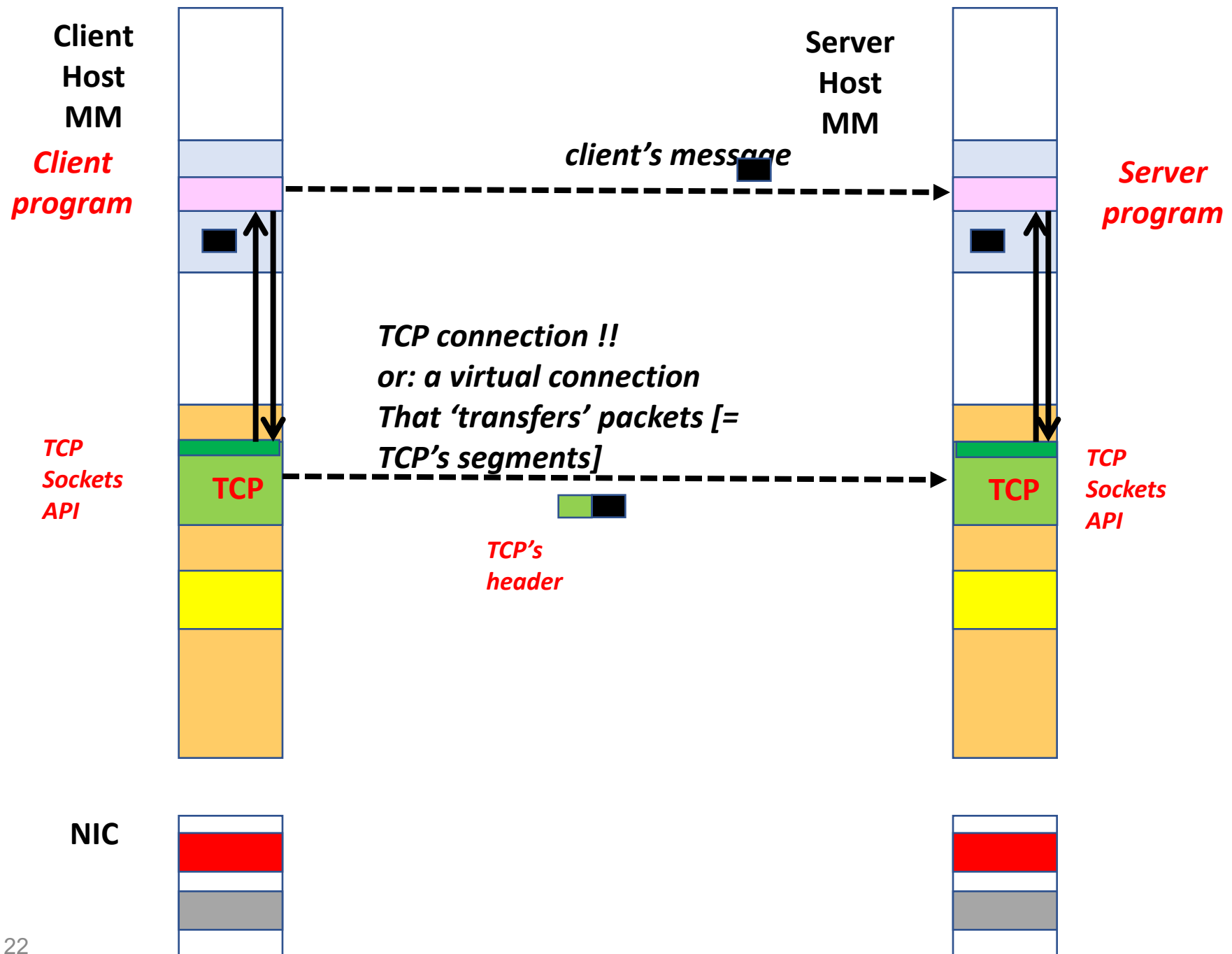
ועל ידי כך:

client program <-----> server program

L5-prot-client <-----> L5-prot-server
socket(), connect(), send(), receive(),
socket(), bind(), listen(), accept(), ...

כל הפונקציות הנ"ל, ועוד, הן חלק מהמכלול הגדול {מקבוצת הפונקציות} שנקרא BSD sockets API
ובשקפים האחרונים 'קשרנו', שוב {!!} את התיאורים הכלליים של רשתות תקשורת ל'עולם' של מהנדסי
תוכנה !!!
לא לעולם של מהנדסי תו"נ.

קישורי TCP – קישורים ווירטואליים



סיכום:

client program <-----> server program

L5-prot-client <-----> L5-prot-server

הודעות

TCP in the client host <-----> TCP in server host

סגמנטים

TCP client <-----> TCP server

מדוע הקישור בין ה- TCP client ל- TCP server מוגדר כ'קישור וירטואלי'?

מכיוון שהוא ממומש על ידי תוכנה.

או: 'אין באמת באופן פיזי' ערוץ כזה

בניגוד לערוצים פיזיים שממומשים 'על באמת'. ע"י חומרה. ומעבירים מידע 'פיזי' [= גלים א"מ]

Notations:

TCP-client & TCP server

TCP-client-sender & TCP-client-receiver

TCP-server-sender & TCP-server-receiver

סיכום ביניים:

TCP מותקן הן במחשב הלקוח והן במחשב השרת

TCP במחשב הלקוח הוא 'חלק' ממכלול הקוד {עשרות מיליוני שורות קוד} של מע' ההפעלה שמותקנת ורצה במחשב הלקוח.

TCP במחשב השרת הוא 'חלק' ממכלול הקוד {עשרות מיליוני שורות קוד} של מע' ההפעלה שמותקנת ורצה במחשב השרת

לצורך המשך הדיון:

נסמן ב- TCP-client את TCP שמותקן ורץ במחשב הלקוח

נסמן ב- TCP-server את TCP שמותקן ורץ במחשב השרת

TCP-client מספק/מממש את כל הפונקציות בממשק הסוקטים גם אלו שבפועל מיועדות לתכניות שרת. היינו, לתכניות שמותקנות ורצות במחשב שרת.

באופן דומה: 'TCP-server מספק/מממש את כל הפונקציות בממשק הסוקטים גם אלו שבפועל מיועדות לתכניות לקוח. היינו, לתכניות שמותקנות ורצות במחשב לקוח.

TCP-client:

TCP-server ל- סגמנטים שולח

וגם: מקבל סגמנטים שנשלחו אליו מ- *TCP-server*

ובאופן דומה:

TCP-server:

TCP-client ל- סגמנטים שולח

וגם: מקבל סגמנטים שנשלחו אליו מ- *TCP-client*

ולכן ל- *TCP-client* יש 2 'חלקים':

TCP-client-sender

TCP-client-receiver

ולכן ל- *TCP-server* יש 2 'חלקים':

TCP-server-sender

TCP-server-receiver

והחלפת הסגמנטים בין TCP-client ל-TCP-server, נעשית 'דרך' קישור TCP
או: נעשית 'דרך' ערוץ ווירטואלי

TCP-client-sender -----> TCP-server-receiver
TCP-client-receiver <----- TCP-server-sender

TCP הוא full duplex:

ז"א דרך אותו קישור ווירטואלי מוחלפים סגמנטים בשני הכיוונים !!

TCP-client-sender -----> TCP-server-receiver
TCP-client-receiver <----- TCP-server-sender

שקפים נלווים לשקפים שעוסקים במאפיינים של פרוטוקול
ה-TCP

או, בניסוח שקול: בשירותים שפרוטוקול ה-TCP מספק

TCP is Connection Oriented

TCP is a Connection Oriented Protocol or: TCP provides a connection oriented service

אחד המאפיינים של TCP הוא העובדה שהוא מוכוון קישור [= Connection oriented]
או, בניסוח אחר: TCP מספק {לפרוטוקולי שכבה 5 ש'משתמשים בו'} שירות שהוא connection oriented

ומה זה אומר?

ה- TCP-client חייב ליזום מול ה- TCP-server שלב מקדמי שנקרא לחיצת יד [= handshake]
בשלב הזה: ה- TCP-client מבקש את אישורו של ה- TCP-server לשלוח לו דאטה {= דאטה שכוללת
הודעות בקשה}

כל עוד ה- TCP-server לא מאשר, ה- TCP-client לא יכול לשלוח לו דאטה !!!!

התיאור הנ"ל, בשקף האחרון: הוא תיאור קונספטואלי. מדויק.
מה פירוש 'קונספטואלי'? תיאור עקרוני שלא מסביר איך בפועל מתכנת מממש לחיצת יד בין תכנית לקוח לבין תכנית שרת!

איך בפועל מתכנת מממש לחיצת יד כזו בין תכנית לקוח לתכנית שרת?
תשובה:

תכנית הלקוח {צד הלקוח של פרוטוקול שכבה 5} חייבת ליזום את לחיצת היד.

על ידי כך שהיא קוראת לפונקציה `connect()`

ז"א המתכנת שמפתח את תכנית הלקוח, ואת צד הלקוח של פרוטוקול שכבה 5, חייב לכלול בקוד שהוא

כותב קריאה לפונקציה `connect()`

שהיא כזכור חלק מממשק הסוקטים. שממומש במלואו על ידי `TCP-client`.

והמתכנת שמפתח את תכנית השרת, ואת צד השרת של פרוטוקול שכבה 5, חייב לכלול בקוד שהוא

כותב קריאה לפונקציה `accept()`

הבאנו אם כן:

תיאור קונספטואלי של אחד מהמאפיינים העיקריים של TCP: העובדה שהוא *connection oriented* ו'לצידו' הבאנו תיאור נוסף {= תיאור 'משלים'} שמיועד למהנדסי תוכנה: איך לממש את לחיצת היד.

והתיאור המשלים ו'אומר': מימו לחיצת היד בין הלקוח לשרת נעשה ע"י כך שתכנית הלקוח קוראת ל-
`connect()` ותכנית השרת, בתגובה, קוראת ל- `accept()`

אין הסבר/תיאור לגבי מה 'מתרחש מאחורי הקלעים' בעקבות הקריאה ל- `connect()`
ז"א: איך `connect()` ממומשת
וכנ"ל אין הסבר/תיאור לגבי מה 'מתרחש מאחורי הקלעים' בעקבות הקריאה ל- `accept()`

או במילים אחרות:
מה עושה ה- `TCP-client-client` כאשר הוא 'מתבקש'/נדרש לבצע `connect()`
ומה עושה 'מולו' ה- `TCP-server-receiver`
בהמשך הקורס, נתאר בתמצית מה 'קורה מאחורי הקלעים'.

ובאופן דומה:
מה עושה ה- `TCP-server-sender` כאשר הוא 'מתבקש'/נדרש לבצע `accept()`
ומה עושה 'מולו' ה- `TCP-client-receiver`
בהמשך הקורס, נתאר בתמצית מה 'קורה מאחורי הקלעים'.

במונחים שלנו:

ה- sender ב- TCP client 'יוזם' את תחילת לחיצת היד.

מול מי? מול ה- receiver ב- TCP server.

נשים לב כי השולח לא יכול 'על דעת עצמו' ליזום לחיצת יד. אלא, הוא עושה זאת 'בתגובה' להנחיה שהוא מקבל 'מגורם שמעליו'.

מי זה הגורם הזה? P5-prot-client

ואיך P5-prot-client 'מנחה' את צד השולח של TCP client להתחיל לחיצת יד? ע"י כך שהוא מבצע קריאה לפונקציה connect() !

תכנית הלקוח היא זו שחייבת ליזום לחיצת יד עם תכנית השרת !!!
לפני בכלל שהיא, בתור לקוח, יכולה לבקש משהו מתכנית השרת.

איך המתכנת 'מתרגם' את האמור לעיל לקוד?
כולל בתכנית הלקוח קריאה ל- `connect()`
או יותר מדויק: בחלק הקוד שמממש את `L5-prot-client` המתכנת כולל קריאה ל- `connect()`

הקריאה ל- `connect()` 'מתחילה את לחיצת היד
'ההמשך' [= האישור. ההסכמה להקים קישור עם הלקוח. ההסכמה לקבל הודעות בקשה מהלקוח. או:
ההסכמה לשרת את הלקוח} חייב להגיע מצד השרת {מהקוד שמממש את `L5-prot-server`

איך האישור 'נשלח' מ- `L5-prot-server` ל- `L5-prot-client`?
הקוד שמממש את `L5-prot-server` חייב לכלול קריאה לפונקציה `accept()`
בקריאה הזו `L5-prot-server` 'מנחה' את `TCP-server` לשלוח אישור {ההסכמה}.
`TCP-client`. ל- מי?

וזו האחרון, 'יעביר' את העובדה שהגיעה הסכמה כזו מצידו של `L5-prot-server` לצד הלקוח של
הפרוטוקול. וע"י כך צד הלקוח של הפרוטוקול 'ידע' שהוא יכול {מותר לו} לשלוח עכשיו בקשות שירות
לצד השרת.

ש: איך `TCP-client` יעביר את האישור ל- `L5-prot-client`?
'באמצעות' ערך החזרה [= ה- `return value`] של הפונקציה `connect()`

TCP is Point-to-Point

TCP is a Point-to-Point Protocol ***or: TCP provides a point-to-point service***

באופן עקרוני פרוטוקול תקשורת יכול לאפשר 3 סוגים {צורות} של העברת מידע:

unicast או: *point-to-point*

העברה בין שולח יחיד למקבל יחיד

multicast

שולח יחיד לקבוצה של מקבלים. לחלק מהקבוצה לה שייך השולח.

Broadcast

שולח יחיד לקבוצה של מקבלים. אבל, בניגוד ל- *multicast* לא רק לחלק מהקבוצה לה שייך השולח, אלא לכל הקבוצה.

פרוטוקול ה- *IP* לדוגמא, מספק את כל 3 הסוגים הנ"ל של העברות מידע.

ובקורס התרכזנו אך ורק בהעברת מידע *unicast* של *IP*

TCP מספק רק סוג אחד של העברת מידע: *unicast* או *point-to-point*

ז"א, אי אפשר באמצעות *TCP* לשלוח סגמנטים שמיועדים ליותר מאשר מקבל אחד.

TCP is Full Duplex

***TCP is Full Duplex Protocol
or: TCP provides a full duplex service***

*TCP מספק שירות של full duplex
מה שאומר שקישור TCP שמוקם בין תכנית לקוח לבין תכנית שרת מאפשר העברת מידע 'דרכו':
גם מצד הלקוח לצד השרת
וגם מצד השרת לצד הלקוח*

TCP provides a byte stream service

נניח שתכנית לקוח (או: צד לקוח של פרוטוקול שכבה 5) מקימה קישור TCP עם תכנית שרת (או: צד השרת של פרוט' שכבה 5)
ונניח כי לאחר הקמת הקישור, תכנית הלקוח שולחת 10 הודעות בסה"כ לתכנית השרת: $m_0, m_1, \dots, m_9$
ולאחר מכן, 'סוגרת' את הקישור.

נניח כי הודעה m_i כוללת m_i -bytes בסה"כ
ז"א ש- TCP במחשב הלקוח, 'התבקש' לשלוח ל- TCP במחשב השרת סך כולל של
 $m_0\text{-bytes} + m_1\text{-bytes} + \dots + m_9\text{-bytes}$

ומבחינת TCP במחשב הלקוח הוא נדרש להעביר byte stream שגודלו הכולל הוא:
 $m_0\text{-bytes} + m_1\text{-bytes} + \dots + m_9\text{-bytes}$

ל- TCP במחשב הלקוח אין 'שום מושג' ש- `m0-byte` הבתים הראשונים מהווים את מה שמבחינת תכנית הלקוח מהווה את ההודעה הראשונה !!
ולכן הוא גם 'לא מודע' לכך שהבתים הבאים, אחרי `m0-bytes` הבתים הראשונים, שייכים להודעה אחרת. ל- `m1`
מבחינתו: הוא התבקש/נדרש להעביר `byte stream` מרגע הקמת הקישור ועד רגע סגירתו.

הערות:

נשים לב כי 'TCP מחלק' את ה- `byte stream` למקטעים שנקראים סגמנטים.
למה? כדי 'להתאים' לעובדה שהרשת הפיזית {פרוטוקולי שכבה 2 ו-1} מעבירה פיזית מידע ממקום למקום על בסיס של שיטת העברה שקרויה מיתוג מנות [= `packet switching`]
ז"א: סגמנט הוא 'מקרה פרטי' של `packet`

הערה נוספת:

תכנית הלקוח לא מעבירה את כל ה- *byte stream* לטיפולו של *TCP* במחשב הלקוח ב'מכה אחת'.
אלא, היא מעבירה את הסך הכולל של הבתים ב'הדרגה'.
או, כפי שנראה בהמשך: ע"י ביצוע קריאות מרובות *{!!!}* לפונק' *send()*
הפונק' *send()* היא חלק מממשק הסוקטים ש- *TCP* 'מספק' לפרוטוקולי שכבה 5

דוגמא:

נניח כי $m0\text{-bytes} = 1000$ בתים
וכנ"ל: $m1\text{-bytes} - m4\text{-bytes}$ בגודל של 1000 בתים כל אחת
ונניח כי $m5\text{-bytes} -- m9\text{-bytes}$ בגודל של 2000 בתים כל אחת
הגודל הכולל בבתים של ה- byte stream ש- TCP במחשב הלקוח מתבקש להעביר {באמינות!} ל- TCP
במחשב השרת הוא לפיכך 15,000 בתים בסה"כ

מבחינת TCP במחשב הלקוח הוא כלל לא מודע לכך ש- 1000 הבתים הראשונים ב- byte stream מהווים
הודעה אחת. הודעה מס' 0.

לא רק זה, אלא גם:
תכנית הלקוח יכולה להעביר את 1000 הבתים ש'מרכיבים' את הודעה $m0$:
ב'פעם אחת'. ז"א, ע"י קריאה פעם אחת ל- $\text{send}()$
בפעמיים. בשלוש וכו'
נתון 'לשיקול דעתו' של המתכנת של תכנית הלקוח.

דבר נוסף: TCP מחלק כאמור את הבתים שמועברים לטיפולו למקטעים שנקראים סגמנטים.
וניה שגודל סגמנט הוא 500 בתים.
אזי 1000 הבתים הראשונים שמהווים את כלל הודעה m_0 ישלחו על ידי TCP במחשב הלקוח ל-TCP
במחשב השרת כלפחות {!!!} שני סגמנטים שונים.
למה לפחות ? נסביר בהמשך.

נראה בהמשך ש- TCP ממספר סידורית את 15,000 הבתים שמהווים את ה- *byte stream*
הבית הראשון מבין ה- 15,000 ממוספר במספר אקראי $\{!!\}$ שנקרא *ISN*
Initial Sequential Number

ז"א מבחינת TCP במחשב הלקוח הוא שולח ל- TCP במחשב השרת 15,000 בתים שממוספרים באופן
הבא:

Bisn, Bisn+1, Bisn+2, Bisn+14,999

נניח שבסגמנט הראשון שנשלח מה- *sender* ל- *receiver* 'ארוזים' 400 בתים.
המספר הסידורי של הבית הראשון בהם הוא *Bisn*

Bisn הוא מספרו הסידורי של הסגמנט הראשון ! לא 0 !

נניח שבסגמנט השני שנשלח מה- *sender* ל- *receiver* 'ארוזים' 300 בתים.

המספר הסידורי של הבית הראשון בהם הוא *Bisn+400*

ו- *Bisn+400* הוא מספרו הסידורי של הסגמנט השני שנשלח מה- *sender* ל- *receiver*. לא 1 !

מספרו הסידורי של הסגמנט השלישי יהיה לפיכך *Bisn+700* ולא 2 !



כל ה- byte stream שנשלח למשל מ- L5-prot-client ל- L5-prot-server
כלל ה- byte stream הזה נשלח מרגע הקמת הקישור בין TCP client ל- TCP server,
עד רגע סגירת הקישור
היינו, מרגע סיום לחיצת היד בין ה- TCP client ל- TCP server, ועד רגע ביצוע `close()`
של הקישור. הן על ידי ה- TCP client והן על ידי ה- TCP server

*TCP provide a RDT service
or: Complete Reliability*

ש: מה פירוש RDT?

ת:

No error

No loss

In-order-delivery

No duplication

הסבר מפורט יותר:

התחייבות להעברת דאטה בצורה אמינה משמעותה להבטיח כי סגמנט {כל סגמנט} שנשלח ע"י ה-sender יגיע ל-receiver לא משובש [= no error]

1. יגיע ל-receiver לא משובש [= no error]

2. לא ילך לאיבוד ב'מהלך' המעבר שלו ברשת בין ה-sender ל-receiver [= no loss]

3. יגיע ל-receiver לפי סדר שליחתו. היינו אחרי כל הסגמנטים שנשלחו לפניו, ולפני כל הסגמנטים שנשלחו אחריו

היינו להבטיח שה-data שנשלח 'בתוך' הסגמנט 'ימסר' לפי מיקומו הנכון ב-byte stream הכולל ליעד הסופי שלו. היינו: פרוטוקול שכבה 5 שמבצע receive() כדי לקבל את ה-data [= in-order-delivery]

4. רק עותק אחד של ה-data שנשלח 'בתוך' הסגמנט 'ימסר' ליעד הסופי שלו. היינו: פרוטוקול שכבה 5 שמבצע receive() כדי לקבל את ה-data [= no duplication]

ש: מדוע עצם שליחת הסגמנטים, בהם 'ארוז' ה- data, לא מבטיחה את כל 4 הנ"ל?
ת: בשל 'מה שקורה בפועל' במהלך המעבר של ה- data בין הצד השולח של TCP לבין הצד המקבל של TCP.

בהמשך, בשקפים הבאים נתאר ביתר פירוט מדוע עצם שליחת סגמנטים ע"י צד שולח של TCP לא מבטיח את הגעתו התקינה [= הלא משובשת] לצד מקבל של TCP

ובדומה נתאר ביתר פירוט מדוע עצם שליחת סגמנטים ע"י צד שולח של TCP לא מבטיח בכלל את הגעתו לצד מקבל של TCP. ז"א לא מבטיח את הגעתו בכלל. גם לא בצורה משובשת.

וכן נתאר ביתר פירוט מדוע עצם שליחת סגמנטים ע"י צד שולח של TCP לא מבטיח את הגעתו לצד מקבל של TCP את הגעתו לפי הסדר 'הנכון' ב- byte stream הכולל של שנשלח מהצד השולח של TCP לצד המקבל של TCP.

וגם נתאר מדוע עצם שליחת סגמנטים ע"י צד שולח של TCP לא מבטיח את הגעתו 'היחידה' לצד מקבל של TCP

מדוע העובדה ש- TCP נדרש לספק שירות RDT היותה
אתגר מורכב מאוד למתכנני הפרוטוקול

או: מדוע התכנון של האופן שבו TCP יממש שירות של
 RDT הוא מאוד מאתגר ורחוק מאוד מלהיות פשוט

אחד האתגרים העיקריים שעמד בפני מתכנני פרוטוקול ה- TCP היה לספק שירות RDT.
ש: למה מדובר באתגר?

ת:

סיבה 1: בשל העובדה ש- TCP 'מקבל' שירות מפרוטוקולים שכולם ללא יוצא מהכלל אינם מספקים
העברה אמינה !!

לא IP, ולא פרוטוקולי שכבה 2 ושכבה 1 !!

סיבה 2: המנגנונים הנדרשים לאפשר RDT {= הכוונה: לאלגוריתמים ולמבני הנתונים הרלבנטיים} –
רחוקים מלהיות טריוויאליים.

ז"א, מכיוון ש- TCP לא 'מקבל' שום עזרה' מהפרוטוקולים ש'מתחתיו' בכל מה שקשור ל- RDT, הוא
'נאלץ' לספק את כל מרכיבי ה- 'RDT בעצמו' !

ובנוסף: הם חייבים להיות לא רק נכונים פונקציונלית אלא גם יעילים. מאוד יעילים.

הבטחת העברת נתונים אמינה [RDT] מחייבת שימוש במנגנון
חיוויים [= acknowledgments indications] בין ה-
sender לבין ה-receiver

מכיוון שעצם שליחת סגמנט מה- *TCP sender* לא מבטיח את 1 עד 4 שתוארו קודם, הרי שעולה מכך כי ה- *TCP sender* חייב לקבל אינדיקציות מה- *TCP receiver* לגבי 'אופן' הגעת כל אחד מהסגמנטים שהוא שולח לו.

ז"א, ה- *TCP sender* שולח ל- *TCP receiver* שבו 'ארוז' *data* {ש'נמסרה' ל- *TCP sender* ע"י פרוטוקול שכבה 5, לצורך העברתה האמינה לצד השני' של פרוטוקול שכבה 5}

הערה: סגמנט כזה שבו 'ארוז' *data* כמתואר לעיל, נקרא **סגמנט דאטה** [*data segment*]

הערה: ישנפ סגמנטים שאינם 'מכילים' *data*. ולכן הם אינם *data segments*.
דוגמא: הסגמנט ש- *TCP client* שולח בעקבות 'הנחיית' *connect()* שהוא מקבל 'מהגורם שמעליו'. מ-
L5-prot-client סגמנט כזה מכיל 0 בתים של דאטה. ז"א סגמנט שכולו רק כותרת *TCP*. וסגמנט כזה

נקרא ***SYN segment***

וכאמור *SYN segment* אינו *data segment*

ז"א ה- *TCP sender* שולח *data segment* ל- *TCP receiver* 'ומחכה לשמוע ממנו' איך הסגמנט הגיע:
תקין או משובש
in-order או *out-of-order*
משוכפל או לא

ש: מה פירוש 'לשמוע' מה- *TCP receiver* איך ה- *data segment* הגיע אליו?

ת: לקבל חיווי מה- *TCP receiver*

או: לקבל אינדיקציה מה- *receiver*

או: לקבל *acknowledge* מה- *receiver*

או: לקבל **סגמנט חיווי** מה- *receiver* [= *acknowledge segment* ובקיצור: *ack segment*]

מה שאומר, מבחינת ה-TCP receiver כי על כל data segment שהוא מקבל מה-TCP sender, הוא חייב לשלוח ack segment מתאים }= כזה ש'יגרום' ל-TCP sender 'להבין' איך ה-data segment שהוא שלח התקבל בפועל!!

כפי שנראה מנגנון החיוויים יספק מענה ל-no error ול-in-order-delivery ול-no duplication
הוא כמובן לא יכול לספק מענה ל-no loss !!

חשוב לשים לב ולהבין:

כמובן שאם סגמנט הלך לאיבוד, הוא כלל לא יגיע ל- TCP receiver !!
מה שאומר כמובן שה- TCP receiver כלל לא ישלח ack segment !

ולכן מנגנון החיוויים לא מתאים כדי לטפל בתסריט של אובדן סגמנט, או יותר מדויק {כפי שנראה בהמשך}: בתסריט של אפשרות {או חשד} לאובדן סגמנט.

מה הכוונה? ה- TCP sender לא יכול בוודאות לדעת אם אכן סגמנט הלך לאיבוד. לכן הוא רק יכול 'לחשוד' שאירוע כזה יתרחש. ואז לתת לו מענה. ע"י שליחה מחדש של הסגמנט שהוא חושד לגביו כי הלך לאיבוד [= לבצע retransmit]

בחלק מהמקרים החשד אכן נכון. היינו הסגמנט אכן הלך לאיבוד.

ואילו בחלק מהמקרים מסתבר, בדיעבד, כי החשד לא היה נכון. וכי הסגמנט לא הלך לאיבוד. ובמקרים

אלו ה- retransmit של הסגמנט 'יגרום' להגעת 2 עותקים או יותר של הסגמנט ל- TCP receiver

עותק אחד הוא של הסגמנט 'המקורי'. זה שלגביו 'התעורר חשד' שהלך לאיבוד.

ועותק שני הוא של הסגמנט שנשלח כדי ב- retransmit.

הערה: במקרה הזה ה- TCP receiver יקבל כמובן duplicate segment. ו'יפעיל' את הפתרון עבור

תסריטים כאלו של הגעת duplicate segments.

*‘Damaged’ Segments
or: Erroneous Segments*

מדוע בכלל סגמנט עשוי להגיע משובש ל- *receiver* ?

סגמנטים עשויים להגיע - receiver 'משובשים'.

ז"א, סגמנט שנשלח ע"י TCP sender, בין אם זה TCP-client-sender או TCP-server-sender, עשוי להגיע משובש!

למי להגיע? ל- TCP receiver

ז"א סגמנט שנשלח למשל ע"י TCP-client-sender עשוי להגיע משובש ל- TCP-server-receiver

למה הסגמנטים יכולים להגיע משובשים?

מכיוון שתוך המעבר של ה- *carrier signal* [= של הגלים הא"מ] בערוצים הפיזיים – ה- *carrier signal* 'חשוף' להפרעות. ל'רעשים' [= *noise*]

מכיוון שסגמנטים שנשלחים למשל מ- *TCP-client-sender* ל- *TCP-server-receiver* עוברים בפועל דרך ערוצים פיזיים, הביטים 'שלהם' יכולים להשתבש תוך כדי המעבר הזה. בשל ה'רעשים' הסביבתיים. ואז ביט שנשלח במקור מה- *TCP-client-sender* כ- '0' עשוי להגיע ל- *TCP-server-receiver* כ- '1' או ככזה שה- *receiver* לא יכול באופן נחרץ לקבוע אם מדובר ב- '0' או ב- '1'.

מהו הפתרון העקרוני לתסריט של הגעת סגמונים משובשים ל-
? receiver

ש: מה המנגנון שעל המתכננים 'לשלב' ב- TCP לצורך מתן מענה לאפשרות הזו של הגעה משובשת של סגמנטים?
ת: Checksum

ש: מה זה checksum?
ת: מעין 'חתימה' {מעין dna} שה- sender מצרף לסגמנטים שהוא שולח.
חתימה של הדאטה שהוא ה- sender שולח ל- receiver

ואז?
כאשר ה- receiver מקבל סגמנט: הוא 'יחשב' בעצמו את החתימה {את ה- checksum} של הדאטה, וישווה את החתימה הזו, שהוא קיבל, עם החתימה {עם ה- checksum} שצורפה לסגמנט. שצורפה ל- header של הסגמנט.

ואם 2 החתימות לא זהות – אזי משמעות הדבר היא שהסגמנט הגיע משובש.
ולכן במקרה כזה ה- receiver חייב 'לזרוק' את הסגמנט [= לא 'להעביר' את התוכן שלו לנמען 'הסופי'.
לפרוטוקול שכבה 5}

ובנוסף, 'לזריקת' הסגמנט שהגיע משובש: לשלוח אינדיקציה מתאימה ל- sender למה? על מנת שה- sender ישלח את הסגמנט פעם נוספת. ז"א: יבצע retransmit

ה- sender לא יכול להבטיח שסגמנט דאטה יגיע תקין ל- receiver.
מה שהוא כן יכול להבטיח הוא שאם סגמנט דאטה יגיע משובש היא ייתן לזה מענה
ש: איך ידע שסגמנט דאטה הגיע משובש?
ת: חישוב checksum + מנגנון החיוויים שה- sender וה- receiver מנהלים ביניהם

ש: איה מענה הוא נותן?
ת: retransmit

בתוך מכלול כלל 'המנגנונים' שמתכנני TCP תכננו כדי לאפשר RDT נכללים המנגנונים הבאים:
מנגנון החיוויים
ה- checksum
ה- retransmit

הערה: אלו לא כל 'המנגנונים' שמתכנני TCP תכננו כדי לאפשר RDT ץ קיימים עוד {הרבה} מנגנונים
נוספים.

כבר 'נתקלנו בעבר ב- `checksum`. בהקשר של הפרוטוקול `UDP`.

אבל יש הבדל מהותי בין ה- `TCP checksum` לבין ה- `UDP checksum`:
במקרה של `UDP` נעשה שימוש {אופציונלי בלבד !!} במנגנון כזה.

ובעיקר: החתימה היא לא של הדאטה שנשלחת 'בתוך' ה- `UDP dg`, אלא מדובר בחתימה של הערכים
של חלק מהשדות של ה- `IP dg`.

להבדיל, במקרה של `TCP`: ה'חתימה הזו' {ה- `checksum`} היא של הדאטה שנשלחת 'בתוך' הסגמנט.

Lost Segments

מדוע בכלל סגמנט עשוי ללכת לאיבוד
וכלל לא להגיע ל- receiver ?

סגמנטים שנשלחים על ידי TCP sender עשויים ללכת לאיבוד 'במהלך המסע שלהם' מה- sending host [= ה- host שבו רץ ה- TCP sender] ל- receiving host [= ה- host שבו רץ ושולח ה- TCP receiver]. היינו, סגמנטים שנשלחים ע"י ה- TCP sender יכולים לא להגיע בכלל ל- TCP receiver.

ש: למה תסריטים כאלו בכלל אפשריים?
ת: בשל עומס לאורך **המסלול** [ה- route] בין ה- sending host ל- receiving host.

ש: מה הכוונה בעומס לאורך המסלול בין ה- sending host ל- receiving host ?
ת: הכוונה לעומס בציודי התקשורת שלאורך המסלול

ש: מה זה עומס בציודי התקשורת?
ת: עומס ב- buffer-ים שהם מנהלים. בתורים שהם מנהלים.
מדובר באותם תורים בהם IP dg שמגיע לציוד התקשורת מאוחסן אחסון זמני עד שהציוד יבצע forwarding שלו 'לעבר'/'כלפי' ה- receiving host

אם קצב ה- forwarding של ציוד תקשורת {של ה- L3 device} הוא קטן מקצב הגעת ה- IP dg ואחסונם {זמנית} בתורים שמנהל ציוד התקשורת, אזי ייתכן כמובן תסריט שבו התורים הולכים ו'מתמלאים'. עד אפס מקום.

מה שאומר כמובן ש- IP dg 'חדשות' שיגיעו לציוד תקשורת כדי שהוא יבצע forwarding שלהם, ילכו לאיבוד !! שהרי אין מקום בתורים לאחסן אותם.

ואם ה- IP dg האלו שהולכים לאיבוד, מכילים סגמנטים, אזי המשמעות היא שלא רק ה- IP dg הלך לאיבוד אלא גם הסגמנט ש'בתוכו' הלך לאיבוד !

מה שמחייב את ה- TCP sender לבצע retransmit של הסגמנט הזה.

מהו הפתרון העקרוני לתסריט של אובדן סגמנטים ?

כפי שתיארנו קודם:

אם סגמנט הלך לאיבוד, הוא כלל לא יגיע ל- TCP receiver !!
מה שאומר כמובן שה- TCP receiver כלל לא ישלח ack segment !

ולכן מנגנון החיוויים לא מתאים כדי לטפל בתסריט של אובדן סגמנט, או יותר מדויק {כפי שנראה בהמשך}: בתסריט של אפשרות {או חשד} לאובדן סגמנט.

ש: מה הכוונה ב'אפשרות'?

ת: ה- TCP sender לא יכול בוודאות לדעת אם אכן סגמנט הלך לאיבוד. לכן הוא רק יכול 'לחשוד' שאירוע כזה יתרחש. ואז לתת לו מענה. ע"י שליחה מחדש של הסגמנט שהוא חושד לגביו כי הלך לאיבוד [= לבצע retransmit]

בחלק מהמקרים החשד אכן נכון. היינו הסגמנט אכן הלך לאיבוד.
ואילו בחלק מהמקרים החשד לא נכון. היינו, הסגמנט לא הלך לאיבוד. ובמקרים אלו ה- retransmit של הסגמנט 'יגרום' להגעת 2 עותקים או יותר של אותו הסגמנט ל- TCP receiver
עותק אחד הוא של הסגמנט 'המקורי'. זה שלגביו 'התעורר חשד' שהלך לאיבוד.
ועותק שני הוא של הסגמנט שנשלח ב- retransmit.

הערה: במקרה הזה ה- TCP receiver יקבל כמובן duplicate segment. ו'יפעיל' את הפתרון עבור תסריטים כאלו של הגעת duplicate segments.

אז מה ה- TCP sender יכול לעשות כדי לתת מענה לתסריטים כאלו?

הוא מעריך [= estimate] את סך הזמן הדרוש ל- data segment שהוא שולח:
להגיע ל- TCP receiver

ל- TCP receiver 'לקלוט' אותו ו'לטפל בו' [= לבדוק אם הגיע תקין, וכן אם הגיע in-order וכו']
ליצור ack segment ולשלוח אותו כחיווי ל- TCP sender

ל- ack segment להגיע לאורך המסלול שבין המארח שבו רץ ה- TCP receiver למארח שבו רץ ה- TCP sender

סך הזמן הזה נקרא בספרות הטכנית: RTT [= Round Trip Time]

ה- TCP sender לא יכול כמובן לחשב במדויק את ערכו של ה- RTT
הוא הרי אפילו לא יודע מהו המרחק {בזמן!} בין ה- *sending host* ל- *receiving host*
הוא גם לא יוע מה המרחק {בזמן!} הדרוש ל- *ack segment* להגיע
ועוד ועוד.

ולכן, ה- TCP sender חייב לבצע הערכה [= estimation] של ה- RTT !
estimatedRTT

ואיך בדיוק? לא טריוויאלי בכלל ! נתאר בהמשך הדיון בנושא TCP. לא בנקודה הזו של התיאור.

ש: בכל אופן, איך ה- TCP sender 'משתמש' בערך $estimatedRTT$?
ת: הוא מכוון timer למשך פרק זמן שגודלו $estimatedRTT$.
ה- timer הוא מנגנון ש'סופר לאחור'.
כאשר ערכו של ה- timer מגיע ל- 0, ה- timer שולח פסיקה [= 'מצלצל']

ה- TCP sender שולח אם כן data segment
מכוון timer למשך פרק זמן שגודלו $estimatedRTT$.
ויש 2 אפשרויות:

א. ack segment ששלח ה- TCP receiver בעקבות הגעתו של ה- data segment – מגיע ל- TCP sender לפני שה- timer 'מצלצל'.

ב. ה- timer 'מצלצל'.
היינו, ack segment לא מגיע ל- TCP sender לפני שה- timer 'מצלצל'.

לגבי אפשרות א': ack segment ששלח ה-TCP receiver בעקבות הגעתו של ה-data segment – מגיע ל-TCP sender לפני שה-timer 'מצלצל':

נשים לב כי ack segment כזה לא בהכרח מציין את הגעתו התקינה של ה-data segment. הוא יכול לציין הגעה משובשת. אבל עצם העובדה שנשלח ack segment 'מעידה' שה-data segment לא הלך לאיבוד.

באופן דומה ה-ack segment לא בהכרח מציין את הגעתו in-order של ה-data segment. הוא יכול לציין הגעה out-of-order. אבל גם במקרה זה, עצם העובדה שנשלח ack segment 'מעידה' שה-data segment לא הלך לאיבוד.

במקרה של אפשרות א', אם כן ה-TCP sender 'מבין' כי ה-data segment שהוא שלח לא הלך לאיבוד ולכן הוא נמנע מביצוע הצעדים שהוא נדרש לבצע במקרה של חשד לאובדן סגמנט.

מה שהוא כן עושה כמובן הוא 'בוחן' את ה-ack segment ו'מגיב' בהתאם. אם ה-ack segment 'מעיד' כי ה-data segment הגיע תקין, ה-sender יבדוק האם ה-ack segment הגיע גם in-order.

אם 2 התנאים מתקיימים: הגיע 'תקין וגם in-order', אזי ה-sender 'מבין' כי הוא עמד בהתחייבותו להעברה אמינה של ה-data שהייתה ארוזה בסגמנט

אם ה-ack segment 'מלמד' את ה-sender כי ה-data segment הגיע תקין אבל out-of-order, אזי ה-sender ישלח את ה-data 'החסר' ל-receiver
הוא ישלח סגמנט עם ה-דאטה החזר, אבל הוא לא ישלח שוב [= יבצע retransmit] את סגמנט הדאטה שהגיע תקין

אם ה-ack segment 'מלמד' את ה-sender כי ה-data segment הגיע משובש, אזי ה-sender ישלח שוב את ה-data segment הזה

לגבי אפשרות ב: ה- timer 'מצלצל'.

היינו, ack segment לא מגיע ל- TCP sender לפני שה- timer 'מצלצל'.

במקרה כזה אין ל- TCP sender 'ברירה' אלא להניח שה- data segment הלך לאיבוד, ולכן לבצע retransmit שלו.

בחלק מהמקרים אכן הסגמנט הלך לאיבוד, ולכן ההנחה של ה- sender 'נכונה'. ופועל יוצא מכך נכון היה שיבצע retransmit

אולם, בחלק מהמקרים הסגמנט לא הלך לאיבוד, ולכן ההנחה של ה- sender 'אינה נכונה'. ופועל יוצא מכך לא נכון היה שיבצע retransmit איך זה ייתכן?

ההערכה שלו את ה- estimatedRTT הייתה 'אופטימית' מדי. בפועל ה- RTT גדול מה- estimatedRTT. אבל ה- sender לא יכול לדעת זאת. הוא יכול כאמור לחשב הערכה בלבד !! במקרה כזה שה- RTT בפועל { ה- RTT 'האמיתי' } גדול מה- estimatedRTT, כמובן שה- timer 'יצלצל' לפני הגעת ה- ack segment

Out-of-order Segments

מדוע בכלל סגמנט עשוי להגיע *out-of-order* ל-*receiver* ?

דוגמא לתסריט אפשרי שמדגימה ייתכנות של הגעה out-of-order:

נניח כי ה- *TCP sender* שולח 3 סגמנטים [*data segments* =]
נסמן אותם לצורך הדוגמא ב- *seg0*, *seg1*, *seg2*
נניח שב- *seg0* ארוזים 'בתים' $m_i, \dots, m_{i+k-1}$ {מתוך ה- *byte stream* הכולל}
נניח שב- *seg1* ארוזים 'בתים' $m_k, \dots, m_{k+q-1}$
נניח שב- *seg2* ארוזים 'בתים' $m_{k+q}, \dots, m_{k+r-1}$

ז"א, ה- *TCP sender* שלח את בתים m_i עד m_{i+r-1} והוא חייב לוודא את הגעתם האמינה
ל- *TCP receiver*
על מנת שה- *TCP receiver* יוכל 'לאחסן' זמני, בחלון הקבלה שהוא מנהל, את הבתים האלו. עד אשר
הנמען {פרוטוקול שכבה 5} עברו הם מיועדים יקרא אותם. יבצע *receive()*

נניח כי:

$seg0$ ו- $seg2$ מגיעים תקינים ל- $TCP\ receiver$.

אבל $seg1$ מגיע משובש. או לא מגיע בכלל.

זאת אומרת, בתים $mi, \dots, mi+k-1$ הגיעו ל- $receiver$ והוא 'אחסן' אותם בחלון הקבלה

וגם בתים $mk+q, \dots, mk+r-1$ הגיעו ל- $receiver$

אבל הוא לא יכול לאחסן אותם בחלון הקבלה 'בצמוד' $\{ = 'מיד אחרי' \}$ בתים $mi, \dots, mi+k-1$

שכן אחרי בתים $mi, \dots, mi+k-1$ ה- $receiver$ חייב לאחסן את בתים $mi, \dots, mi+k-1$

ורק אחריהם, אחרי בתים $mi, \dots, mi+k-1$, הוא 'יופיעו' בתים $mk+q, \dots, mk+r-1$

לכן, ה- $receiver$ שומר את $mi, \dots, mi+k-1$ בחלון הקבלה.

ו'משאיר' בחלון הקבלה מקום פנוי $\{ 'חור' \}$ בגודל q בתים $\{$ בהם ישמרו בהמשך בתים $mk, \dots, mk+q-1$.

מתי? כאשר הם יגיעו תקינים ל- $receiver$

ו'אחרי' במקום הפנוי הזה ה- $receiver$ ישמור את בתים $mk+q, \dots, mk+r-1$



כל ה- byte stream שנשלח למשל מ- L5-prot-client ל- L5-prot-server
כלל ה- byte stream הזה נשלח מרגע הקמת הקישור בין TCP client ל- TCP server ,
עד רגע סגירת הקישור
היינו, מרגע סיום לחיצת היד בין ה- TCP client ל- TCP server , ועד רגע ביצוע close()
של הקישור. הן על ידי ה- TCP client והן על ידי ה- TCP server



מכלול הבתים של ה- *byte stream* נשלח על ידי *TCP sender* במחשב הלקוח כאוסף של סגמנטים.

מבין כלל הסגמנטים שנשלחים, וביחד 'מעבירים' את כלל ה- *byte stream*, הציור מראה 'מקטעים' של ה- *byte stream* המקטע האפור, הסגול והתכלת כל אחד מהמקטעים האלו 'נארז' ונשלח בסגמנט נפרד. היינו *TCP sender* שולח 3 סגמנטים ש'ביחד' מעבירים את סך הבתים שב- 3 המקטעים. 3 סגמנטים עוקבים.

נניח כי:

seg0 הוא הסגמנט ש'בתוכו' נארז מקטע הדאטה האפור.

היינו 'בתוכו' נארזים בתים $m_i, \dots, m_{i+k-1}$

seg1 הוא הסגמנט ש'בתוכו' נארז מקטע הדאטה הסגול.

היינו 'בתוכו' נארזים בתים $m_k, \dots, m_{k+q-1}$

seg2 הוא הסגמנט ש'בתוכו' נארז מקטע הדאטה התכלת.

היינו 'בתוכו' נארזים בתים $m_{k+q}, \dots, m_{k+r-1}$



נניח כי:

$seg0$, הסגמנט ש'בתוכו' נארז מקטע הדאטה האפור מגיע תקין ל- $receiver$ היינו, בתים $mi, \dots, mi+k-1$ מגיעים ל- $receiver$ והוא שומר אותם בחלון הקבלה שהוא מנהל.

כנ"ל ל $seg2$, הסגמנט ש'בתוכו' נארז מקטע הדאטה התכלת מגיע תקין ל- $receiver$ היינו בתים $mk+q, \dots, mk+r-1$ מגיעים ל- $receiver$ והוא שומר אותם בחלון הקבלה שהוא מנהל.

אבל לא 'בצמוד' לבתים האפורים !

$seg1$, הסגמנט ש'בתוכו' נארז מקטע הדאטה הסגול לא מגיע תקין ל- $receiver$ =] מגיע משובש] או שלא מגיע בכלל [= הלך לאיבוד] היינו בתים $mk, \dots, mk+q-1$ כלל לא מגיעים ל- $receiver$ היינו 'נוצר חור' בחלון הקבלה. הבתים האפורים, אחריהם חור, ואחריהם הבתים בצבע תכלת.

לפיכך פרוטוקול שכבה 5 שקורא, מבצע $receive()$, יכול 'לקבל' את הבתים האפורים, אבל לא יכול לקבל את הבתים התכולים כל עוד הבתים הסגולים לא יגיעו תקינים ל- $receuwer$ והוא ישמור אותם {במיקומם הנכון !} בחלון הקבלה !



לאחר שבתים $mk, \dots, mk+q-1$ יגיעו תקינים ל- receiver, בשל ה- retransmit שביצע
ה- sender של seg1, אזי ה'חור' שנוצר קודם בחלון הקבלה 'ייסגר'
וה- L5-prot-server יכול עכשיו לקרוא את הבתים הסגולים ואת הבתים התכולים
זאת כמובן לאחר שהוא קרא את הבתים האפורים.

מהו הפתרון העקרוני לתסריט של הגעת סגמנטים *out-of-order*?

תואר לעיל, כחלק מהתיאור שמסביר מדוע בכלל ייתכן כי *data segments* שנלשחים על יד *TCP sender* עשויים להגיע *out-of-order* ל-*receiver* היינו, אחסון 'עם חורים' בחלון הקבלה.

Duplicated Segments

מדוע בכלל מספר עותקים של אותו סגמנט
עשויים להגיע ל- receiver ?

בדיון הקודם בנושא של *lost segments*, תיארנו כי הפתרון אותו מממש ה- *TCP sender* כדי לתת מענה לתסריטים כאלו מבוסס על הערכה של גודל *estimatedRTT* ושימוש ב- *timer*

ה- *TCP sender* שולח *data segment* מכוון *timer* למשך פרק זמן שגודלו *estimatedRTT*.

וראינו כי יש 2 אפשרויות.
ש אחת מהן {סימנו אותה בדיון הנ"ל כאפשרות ב'} היא שה- *timer* 'מצלצל'.
היינו, *ack segment* לא מגיע ל- *TCP sender* לפני שה- *timer* 'מצלצל'.

והסברנו כי במקרה כזה אין ל- *TCP sender* 'ברירה' אלא להניח שה- *data segment* הלך לאיבוד, ולכן לבצע *retransmit* שלו.
בחלק מהמקרים אכן הסגמנט הלך לאיבוד, ולכן ההנחה של ה- *sender* 'נכונה'. ופועל יוצא מכך נכון היה שיבצע *retransmit*

אולם, בחלק מהמקרים הסגמנט לא הלך לאיבוד, ולכן ההנחה של ה- *sender* 'אינה נכונה'. ופועל יוצא מכך לא נכון היה שיבצע *retransmit*

ז"א יתכן כי ה- *sender* שולח *data segment* שנסמן אותו כ- *segJ* הסגמנט מגיע תקין ל- *receiver*.

שכמובן 'ישלוף' מתוכו את ה- *data* ש'ארוז' בו וישמור את ה- *data* במקום המתאים בחלון הקבלה

אבל מכיוון שה- *estimatedRTT* היה קטן מדי {'אופטימי מדי'} ביחס ל- *RTT* ה'אמיתי', ה- *timer* 'צלצל'.
מה שאילץ את ה- *sender* לבצע *retransmit* של *segJ*
ונניח כי גם עותק זה, העותק השני, של *segJ* מגיע תקין ל- *receiver*
ז"א, ייתכן כי 2 עותקים תקינים של *segJ* יגיעו תקינים ל- *receiver*

כאשר ה- receiver מקבל *data segment* תקין הוא חייב להיות מסוגל לקבוע האם זו 'הפעם הראשונה' שהוא מקבל את הסגמנט, או שהוא 'כבר קיבל בעבר' עותק שלו.

ש: למה חייב?

ת: כי הוא חייב 'להתעלם' מהעותקים של *data segment* שהוא כבר קיבל 'בעבר'.
ז"א, הוא חייב 'לזרוק' עותקים אלו.

או: 'אסור לו' לשלוף את ה- *data* ש'ארוזה' ב- *duplicated segments* ולשמור אותה בחלון הקבלה !!

מהו הפתרון העקרוני לתסריט של הגעת *duplicate segment*?

שימוש במספרים סידוריים.
לכל data segment שנשלח על ידי ה- snede 'מצורף' מספר סידור.

ש: איך המספר הסידורי מצורף ל- data segment?
ת: לכותרת [=header] של הסגמנט.

Control Flow

מה זו בכלל בקרת זרימה $[flow\ control =]$
ולמה בכלל לא פשוט לתכנן מנגנון כזה שהוא גם 'נכון' וגם יעיל ?

המונח *flow control* מתורגם בעברית ל- 'בקרת עומסים'

TCP מממש בקרת עומסים

זה לא *RD*

אז למה הכוונה?

כאשר דאטה {סגמנטים ש'מכילים' דאטה [חלק מהודעה של שכבה 5]} מגיע ל- *TCP receiver*, וה-
receiver בדק וזיהה כי הדאטה הגיעה תקינה ובנוסף הגיעה *in-order*, אזי ה- '*receiver* שולף' את
הדאטה מתוך הסגמנט ושומר באופן זמני 'אצלו'.
איפה? ב- *buffer*. שנקרא: **חלון קבלה** [*receive window*]

ש: למה הוא עושה זאת?

ת: כדי שהדאטה תהיה זמינה לפרוטוקול שכבה 5 כשזה 'יחליט' לקבל אותה. לקרוא אותה. לבצע
receive() {שהיא חלק מממשק הסוקטים}

במילים אחרות הדאטה מגיעה ל- *TCP receiver*, בנקודת זמן t_1 , באופן לא מסונכרן {!!} עם הבקשה של פרוטוקול שכבה 5 לקרוא את הדאטה.

לא מסונכרן עם הנק' בזמן t_2 , שבה פרוטוקול שכבה 5 מבצע *receive()*

לכן ה- *TCP receiver* חייב לשמור זמנית את הדאטה לאחר הגעתה התקינה בזמן t_1 , ועד זמן t_2 שבה פרוטוקול שכבה 5 יקרא אותה

ה- *buffer* הזה [= ה- *receive window*] הוא סופי בגודלו.

ולכן ייתכן התסריט הבא: קצב שליחת הדאטה על ידי ה- *TCP sender* הוא מהיר.

'הרבה' דאטה מגיעה ל- *TCP receiver*. והוא 'צובר' עוד ועוד דאטה ב- *receive window*

ובמשך פרק הזמן הזה קצב הקריאה של הדאטה ע"י פרוטוקול שכבה 5 {=} קצב ה'ריקון' של חלון הקבלה}, הוא איטי יותר

במצב עניינים כזה חלון הקבלה 'יתמלא' בהדרגה. יותר ויותר. עד שיהיה מלא לחלוטין.

מה שאומר, שדאטה חדשה שתגיע ל- *TCP receiver* מה- *TCP sender* תלך לאיבוד {!!!} – מכיוון שאין מקום לאחסן אותה בחלון הקבלה.

למרות שהיא תקינה לחלוטין ומגיעה *in-order* – היא הולכת לאיבוד !

מה שייאלץ את ה- *TCP sender* לבצע *retransmit* !!

בשל כך:

משך הזמן הכולל הדרוש ל- *TCP sender* להעביר ל- *TCP receiver* את כל ה- *byte stream* – יתארך !!

ודבר נוסף: ה- *retransmit* יוסיף עוד עומס על רשת התקשורת !! – וגידול בעומס על רשת התקשורת הוא בעייתי !

ש: למה 'בעייתי'?

ת: מכיוון שה- *retransmit* יצריך שימוש במשאבי הרשת.

ש: אלו משאבים?

ת: ערוצי התקשורת הפיזיים. וכן עומס על ציודי התקשורת. ועל ה- *buffer*-ים שהם מנהלים. ועוד ועוד.

המסקנה מהדיון עד כה:

אם קצב השליחה של דאטה ע"י ה- *TCP sender* הוא גבוה מקצב ה'ריקון' של חלון הקבלה ע"י הנמען $\Rightarrow$ פרוטוקול שכבה 5 – אזי דאטה תלך לאיבוד.
דבר שיצריך שליחה מחדש. ש'תביא איתה' מגוון בעיות.

לכן, חשוב מאוד ל'השתדל' להימנע מתסריטים כאלו.
המנגנון שמטרתו/תפקידו להימנע מתסריטים כאלו נקרא **בקרת זרימה** [*flow control*]
'בגדול' האופן שבו המנגנון הזה עובד: הוא גורם ל- *TCP sender* 'לווסת' את קצב השליחה שלו בהתאם
לכמות המקום הפנוי שנותרה בחלון הקבלה!
'לווסת' משמעו: להקטין או להגדיל את קצב השליחה

ש: מתי להקטין?

ת: כאשר המקום הפנוי בחלון הקבלה הולך ומצטמצם. שזה קורה כאמור כאשר קצב 'ריקון' של חלון
הקבלה הוא איטי ביחס לקצב השליחה וההגעה.

ש: מתי להגדיל?

ת: כאשר המקום הפנוי בחלון הקבלה הולך וגדל.

בקרת זרימה:

ה- receiver 'מעדכן' באופן שוטף את ה- sender לגבי כמות המקום הפנוי שנותר בחלון הקבלה והעדכונים השוטפים האלו 'נלקחים בחשבון' ע"י ה- sender ו'מווסתים' את קצב השליחה שלו

הערה:

קצב שליחה איננו כמות הבתים שיש לשלוח

את זה ה- sender לא יכול 'לקבוע'. זה נקבע ע"י פרוטוקול שכבה 5.

קצב השליחה אם כן לא מתייחס לכמות הבתים, אלא 'עד כמה מהר' הם ישלחו ע"י ה- sender

Congestion Control

מה זה בכלל בקרת עומסים [=congestion control]
ולמה בכלל לא פשוט לתכנן מנגנון בקרת עומסים
שהוא גם 'נכון' וגם יעיל ?

בנוסף לצורך לתכנן מנגנון בקרת שליחה, עמד בפני מתכנני פרוט' ה- TCP אתגר {גדול עוד יותר!} והוא לתכנן מנגנון בקרת עומסים [= congestion control]

ש: מה זה מנגנון בקרת עומסים?

ת: מנגנון שיווסת את קצב שליחת הדאטה, על ידי ה- TCP sender בהתאם לעומס הנוכחי ברשת.
מנגנון שנועד להבטיח כי ככל שהעומס ברשת גדול יותר, כך קצב השליחה של הסגמנטים ע"י ה- TCP sender יהיה קטן יותר !!

ש: מה הכוונה בעומס על הרשת?

ת: הכוונה לעומס בציודי התקשורת של אורך המסלול [ה- route] שבין ה- sending host ל- receiving host

ה- sending host -- אותו ה- host שבו רץ ושולח ה- TCP sender

ש: מה זה עומס בציודי התקשורת?

ת: עומס ב- buffer-ים שהם מנהלים. בתורים שהם מנהלים.
אלו אותם תורים בהם דאטה שמגיע לציוד התקשורת מאוחסן אחסון שמני עש שהציוד יבצע forwarding שלו 'לעבר' ה- destination host הנכון.

אם קצב ה- forwarding של ציוד תקשורת {של ה- L3 device} הוא גבוה מקצב הגעת ה- IP dg ואחסונם {זמנית} בתורים שמנהל ציוד התקשורת, אזי ייתכן כמובן תסריט שבו התורים הולכים ו'מתמלאים'. עד אפס מקום.

מה שאומר כמובן ש- IP dg 'חדשות' שיגיעו לציוד תקשורת כדי שהוא יבצע forwarding שלהם, ילכו לאיבוד !! שהרי אין מקום בתורים לאחסן אותם.

ואם ה- IP dg האלו שהולכים לאיבוד, מכילים סגמנטים, אזי המשמעות היא שלא רק ה- IP dg הלך לאיבוד אלא גם הסגמנט ש'בתוכו' הלך לאיבוד !

מה שיחייב את ה- TCP sender לבצע retransmit של הסגמנט הזה.

מה שיגדיל את משך הזמן הכולל הדרוש ל- TCP sender להעביר ל- TCP receiver את כל ה- byte stream – יתארך !!

וכמו כן, ה- retransmit יוסיף עוד עומס על רשת התקשורת !!

וכבר הסברנו כי גידול בעומס על רשת התקשורת הוא בעייתי !

בקרת עומסים:

ניטור שוטף של העומס ברשת

מבחינת ה- sender התאמה, או וויסות, של קצב השליחה של data segments בהתאם.
עומס נמוך יחסית – קצב שליחה אפשרי גבוה {יחסית}
עומס גבוה ברשת – האטה של קצב השליחה

הערה:

לגבי בקרת זרימה, ה- receiver 'מעדכן' באופן שוטף את ה- sender לגבי העומס 'אצלו'. היינו, העומס בחלון הקלה שהוא מנהל. ז"א, כמות המקום הפנוי שנותר בחלון הקבלה.
לגבי בקרת עומסים – כמובן לא רלבנטי שה- receiver 'ילמד' את ה- sender לגבי העומס הנוכחי.
ה- sender 'לומד' את זה בעצמו'. ע"י ניטור הרשת.
וזה לכשעצמו מאוד לא פשוט !! כפי שמתואר, ממש בקצרה, בשקפים הבאים.

מה האתגרים הגדולים כאשר באים לתכנון מנגנון בקרת עומסים?

בכל רגע נתון בזמן ניתן לשייך מדד שמייצג את העומס הנוכחי ברשת תקשורת. ואם הערך של המדד הזה הוא גבוה, ז"א קיים עומס גבוה ברשת, חייבים להימנע מהגדלה נוספת של העומס.

למה? כדי לא 'לחנוק' את הרשת או במילים אחרות: כדי למנוע מהזמן ההמתנה שדרוש להעביר מידע ברשת – למנוע לגדול ולגדול. בטח למנוע גידול אקספוננציאלי.

ולכן, כאשר 'מזהים' עומס גבוה, אזי חובה על ה-sender להאיט את קצב השליחה לרשת. או, במילים אחרות: להקטין את כמות הדאטה שהם 'מוספים' לרשת {כדי שהרשת תעביר אותה}

אז אם כך מנגנון 'בקרת עומסים' הוא מנגנון שחייב לנטר באופן שוטף [= 'כל הזמן'] את העמוס ברשת.
מה שמוסיף תקורה [= overhead] לזמן שדרוש להעביר את ה-byte stream
ולכן לממש מנגנון יעיל [= כזה שהתקורה שהוא מוסיפה היא 'סבירה' {= היא לא overkill} – רחוק
מלהיות טריוויאלי

ולא רק זה, אלא שהמנגנון צריך ל'תרגם' את העומס הנוכחי ברשת לערך מספרי!
וגם למצוא קונספטואלית ערך כזה ש'מבטא' בצורה טובה את העומס הנוכחי ברשת הוא לא פשוט.

הערות:

1. TCP מממש מנגנון כזה !!
 2. בנוסף למנגנון בקרת הזרימה !!
 3. כל TCP sender-ים מבצע את הניטור השוטף של העומס ברשת, ומתרגם אותו לערך מספרי יחיד
{שנקרא cwin. קיצור של congestion window}.
- והוא 'מתחשב' בערך הזה **בבואו** ל'קבוע' את קצב השליחה לרשת.

סיכום: האתגרים שעמדו בפני מתכנני פרוטוקול ה-TCP

סיכום: האתגרים שעמד בפני מתכנני ה- TCP היו מאוד מורכבים ולא פשוטים.

1. לגבי המנגנון שמיועד להבטיח RDT - הוא עצמו 'מורכב' ממספר תתי מנגנונים.
אחד שמטפל בתסריטי הגעה של דאטה משובשת.
ושני שמטפל בתסריט של אובדן דאטה.
ושלישי שמטפל בתסריט של הגעה 'משוכפלת' של דאטה.
ורביעי שמטפל בהגעה out-of-order
וכל מנגנון כזה בפני עצמו מורב ולא פשוט
2. ולצד מנגנוני ה- RDT השונים: מנגנון בקרת זרימה
3. ובנוסף: מנגנון בקרת עומסים

כאמור כל אחד מהמנגנונים האלו בפני עצמו – לא פשוט לתכנון.
בטח ובטח השילוב ביניהם.

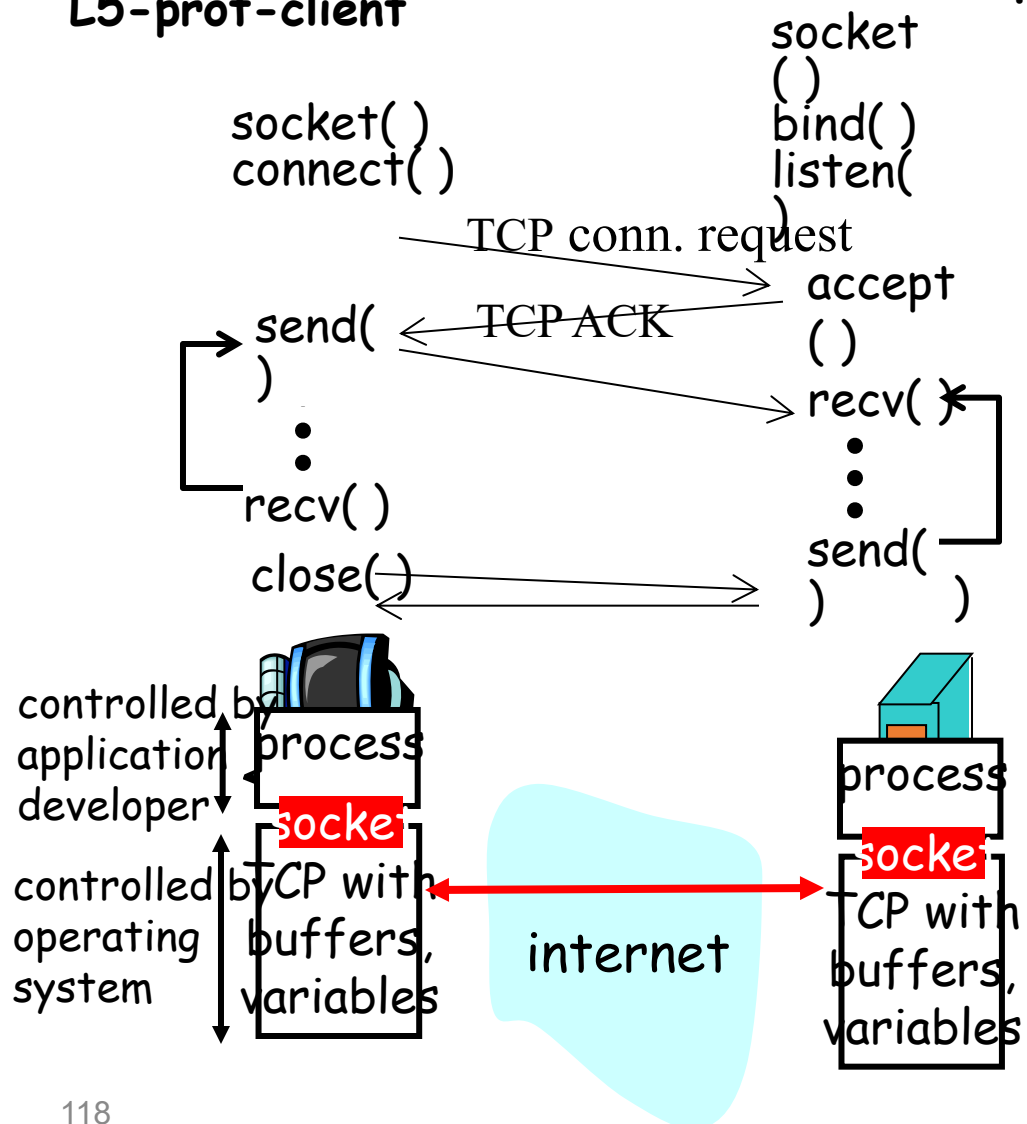
ואם 'זה לא מספיק' – ההכרח שהם יעילים וגם השילוב שלהם יהיה יעיל !!

Client-Server – Overall Interaction

Socket-programming using TCP

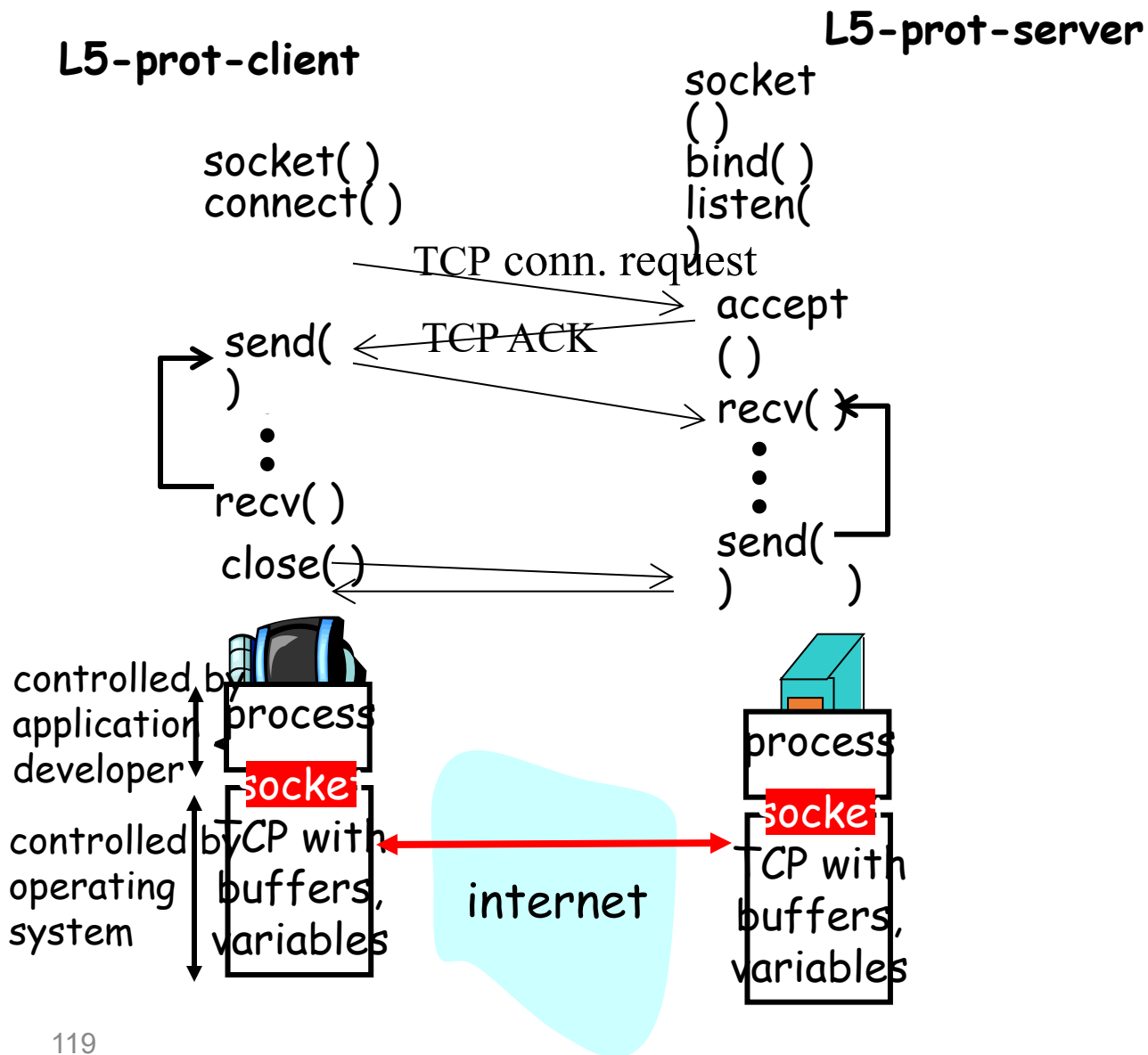
L5-prot-client

L5-prot-server



- השקף מתאר את האינטראקציה
(הכוללת !) בין תכנית לקוח לבין
תכנית שרת
- או יותר מדויק: בין L5-prot-client לבין
L5-prot-server

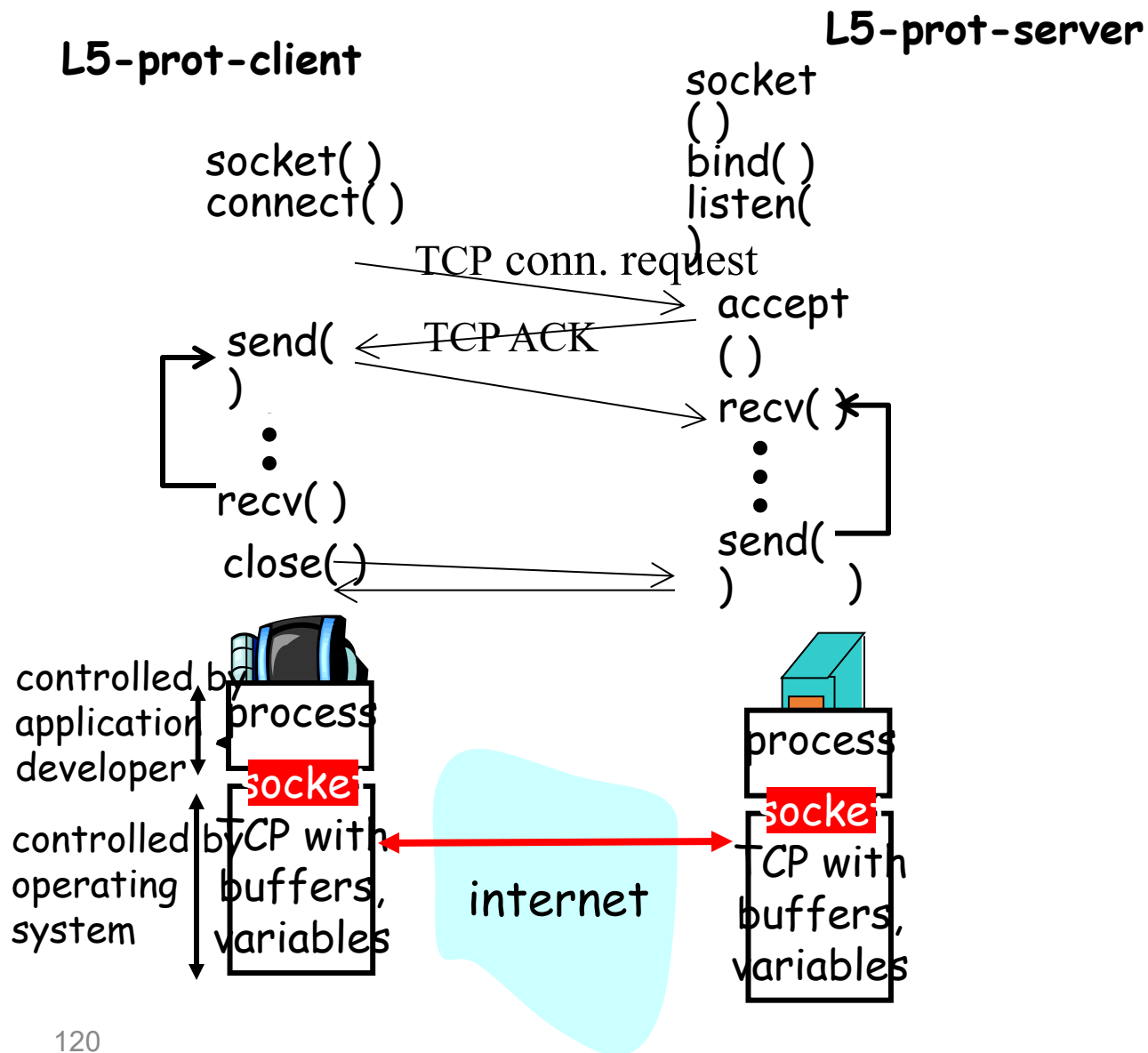
Socket-programming using TCP



- האינטראקציה נעשית ע"י כך ש-
 L5-prot-client קורא פונקציות [=]
 מבצעות קריאות מערכת
 - לדוגמא: socket(), bind(), ...
 - פונקציות אלו מהוות חלק מממשק
 הסוקטים

- וכנ"ל לגבי L5-prot-server

Socket-programming using TCP



- המימוש של כל הפונקציות האלו
(socket, bind, listen, ...) נעשה ע"י

מודול ה-TCP

- היינו, הפונקציות האלו הן חלק
ממאות (רבות) של פונקציות ש'ביחד'
מהוות את המימוש של פרוטוקול ה-

TCP

- שהוא 'חלק' מעשרות ומאות אלפי
פונקציות שביחד מהוות חלק {קטן

יחסית!} מכלל הקוד של מערכת

ההפעלה

Ports & Sockets

TCP מספק שירות קצה-לקצה

מה ש'מחייב אותו' ל'הגדיר' מזהה של תכנית

וההגדרה שניתנה ע"י מתכנני מודל ה- TCP/IP עבור 'מזהה תכנית' היא: **port**

המושג port הוא מושג אבסטרקטי. קונספטואלי. שנועד לציין מזהה תכנית

ומה עם הרובד 'התוכנית'? האובייקט 'התוכנית המקביל' לאובייקט port נקרא סוקט
סוקט הוא המונח הנרדף שנתנו מתכנני ממשק הסוקטים {ה- BSD sockets} למונח port

תכנית הלקוח רצה במחשב. מחשב הלקוח. והריצה שלה מנוהלת ע"י מ"ה שמותקנת ורצה במחשב הלקוח. למשל: ווינדוז, או לינוקס, או אנדרואיד וכו'.
ועובדה זו פירושה: שמ"ה במחשב הלקוח מקצה 'מזהה' לתכנית הלקוח.
מזהה זה נקרא process id

ש: אם כך, אם לתכנית הלקוח כבר מוקצה מזהה ע"י מערכת ההפעלה במחשב שבו היא רצה, אז מדוע מצאו מתכנני מודל ה- TCP/IP לנכון לזהות תכנית ע"י מזהה אחר ?? ע"י ה- port?
ז"א, מודע הם לא בחרו 'להשתמש' במזהה התכנית שממילא ניתן לה ע"י מ"ה?

תשובה לשאלה הזו:
ה- process id ספציפי למערכת הפעלה.
סוג זה של זיהוי שונה, למשל, בין ווינדוז לבין לינוקס.
ומתכנני מודל ה- TCP/IP היו חייבים לפיכך להגדיר מזהה תכנית שאיננו 'תלוי מערכת הפעלה'.

בכל מחשב שמשויך לרשת הווירטואלית חייב להיות מותקן TCP !!!
ולכן TCP מותקן גם במחשבי ווינדוז, וגם במחשבי לינוקס וכו'.
ומכאן ניתן להבין את ההכרח שעמד בפני מתכנני המודל להגדיר מושג 'כללי' [= כזה שלא תלוי במ"ה כזו
אחרת] ש'ישמש' כמזהה תכנית 'גנרי'.
והם אכן עשו זאת.
והשם שהם בחרו 'לתת' למזהה הכללי הזה הוא כאמור port
ומתכנני ממשק הסוקטים בחרו לעשות שימוש במונח {או בשם} סוקט במקום במונח port.

Ports [= Sockets] Characteristics

ל- port [= לסוקט] יש מספר מאפיינים:

מאפיינו ראשון הוא **סוג ה-port** או: סוג הסוקט
האם מדובר בפורט של TCP או של UDP
שהרי 2 הפרוטוקולים האלו מספקים שירות העברה קצה-לקצה
ולכן, כל אחד מהם, חייב ל'ספק' מזהה תכנית.
היינו: פורט.
מה שאומר ש'חלק' מהפורטים הם כאלו שמשמשים את TCP וחלק את UDP

מאפיין שני של אובייקט מסוג פורט {סוקט} הוא: **מספר פורט**.

ערכו של המאפיין הזה הוא מספר שלם אי שלילי.

ז"א גדול או שווה ל-0.

וקטן מ-1 - 2^{16}

ז"א מספר פורט הוא מס' שלם בתחום שבין 0 ל-65,535

ש: למה דווקא התחום הזה?

ת: מכיוון שמספר פורט מיוצג, בינארית, על ידי 16 ביטים.

היות ול-16 הביטים האלו 'מתייחסים' [= נותנים משמעות] של מספר שלם חסר סימן, נובע מכך שמספר

פורט הוא בין 0 ל-65,535

ש: למה בכלל צריך מאפיין כזה?
ת: בכל רגע נתון בזמן יכולות לרוץ בו זמנית במארח מסוים, H , מספר תכניות שהן חלק מאפליקציה רשתית.

למשל: מספר ריצות שונות של תכנית *browser*
או מספר ריצות של תכנית *browser* ו'לצידן' ריצה של תכנית ה- *outlook*
ותוכניות רשתיות נוספות

כל זה כאמור באותו המארח H
 TCP שמותקן ורץ ב- H חייב לזהות כל אחת מהתכניות הרשתיות שרצות בו באמצעות מזהה ייחודי.
ז"א אם בנק' זמן מסוימת t , רצות ב- H התכניות הרשתיות $P_0, P_1, \dots, P_{n-1}$, אזי TCP ב- H חייב להבחין/להבדיל ביניהן
ואיך הוא עושה זאת? ע"י מספר הפורט
ז"א מספר הפורט ש- TCP ב- H 'קצה' לצורך זיהוי תכנית P_0 יהיה שונה ממספר הפורט שהוא יקצה לצורך זיהוי של תכנית P_1

במילים אחרות: המאפיין הזה של מספר פורט נועד להבחין בין תכניות שונות שרצות באותו מארח

נשים לב שאם 2 תכניות Pa - Pb רצות במארחים שונים, $H1$ ו- $H2$, אזי לשתיהן ייתכן אותו מספר פורט
אז איך 'מבדילים' ביניהן ברשת הווירטואלית הגדולה לה הן שייכות?
על ידי:

$\langle IP\text{-}addr\text{-}H1, \text{port number } Pa \rangle$

$\langle IP\text{-}addr\text{-}H2, \text{port number } Pb \rangle$

במילים אחרות: המאפיין הזה של מספר פורט הוא מזהה 'בתוך' מארח. לא בין מארחים.

ולכן, 'לרשות' כל מארח 'עומד מאגר' של $2^{16}-1$ פורטים

או יותר מדויק: 'לרשות' TCP ביחד עם UDP שמותקנים ורצים באותו מארח, H , מארח 'עומד מאגר' של $2^{16}-1$ פורטים

עד עכשיו ראינו 2 מאפיינים של אובייקט הפורט {הסוקט}
סוג הפורט
מספר פורט מקומי [=local port number]

קישור TCP מוקם ומנוהל בין 2 צדדים {!!}
בין TCP-client לבין TCP-server

וקישור כזה 'משמש' להחלפת מידע באופן דו-כיווני {!!} [full duplex] :
בכיוון אחד: בין TCP-client-sender לבין TCP-server-receiver
ובכיוון השני: בין TCP-server-sender לבין TCP-client-receiver

ודו-כיווניות זו 'מייצגת' החלפת מידע דו-כיווני בין L5-prot-client ובין L5-prot-server
והרי L5-prot-client הוא 'חלק' מתכנית הלקוח. שחייבת להיות מזוהה במחשב הלקוח. על ידי מספר
פורט
ובדומה: L5-prot-server הוא 'חלק' מתכנית השרת. שחייבת להיות מזוהה במחשב השרת. על ידי מספר
פורט.

אם כך לפורט שמזהה את L5-prot-client יש בנוסף לשני המאפיינים אותם הזכרנו קודם {סוג הפורט ומספר הפורט המקומי} מאפיין נוסף. שלישי. והוא: **מספר פורט מרוחק** [*remote port number* =] ומספר הפורט המרוחק מזהה את 'הקצה השני' של הקישור.

זאת אומרת הפורט שמזהה את תכנית הלקוח {במחשב הלקוח !!} מאופיין ע"י:

סוג פורט

מס' פורט מקומי. זה שמזהה את תכנית הלקוח במחשב הלקוח

מס' פורט מרוחק. זה שמזהה את 'הקצה השני' של הקישור. ז"א, זה שמזהה את 'התכנית בצד השני של הקישור'. ז"א, את תכנית השרת. שרצה במחשב השרת.

ברור שבאופן סימטרי לחלוטין מתקיים:

הפורט שמזהה את תכנית השרת {במחשב השרת !!} מאופיין ע"י:

סוג פורט

מס' פורט מקומי. זה שמזהה את תכנית השרת במחשב השרת

מס' פורט מרוחק. זה שמזהה את 'הקצה השני' של הקישור. ז"א, זה שמזהה את 'התכנית בצד השני של הקישור'. ז"א, את תכנית הלקוח. שרצה במחשב הלקוח.

ועד כאן: כל פורט {סוקט} מזוהה על ידי 3 מאפיינים {פרמטרים}

ויש 2 נוספים:

local IP address – כתובת ה- IP של המארח שבו רצה התכנית שמזוהה על ידי ה- *local port*
– *remote IP address* כתובת ה- IP של שבו 'רצה התכנית השניה'. זו שמזוהה על ידי ה- *remote port*

ועד כאן: כל פורט {סוקט} מזוהה על ידי 3 מאפיינים {פרמטרים}

ויש 2 נוספים:

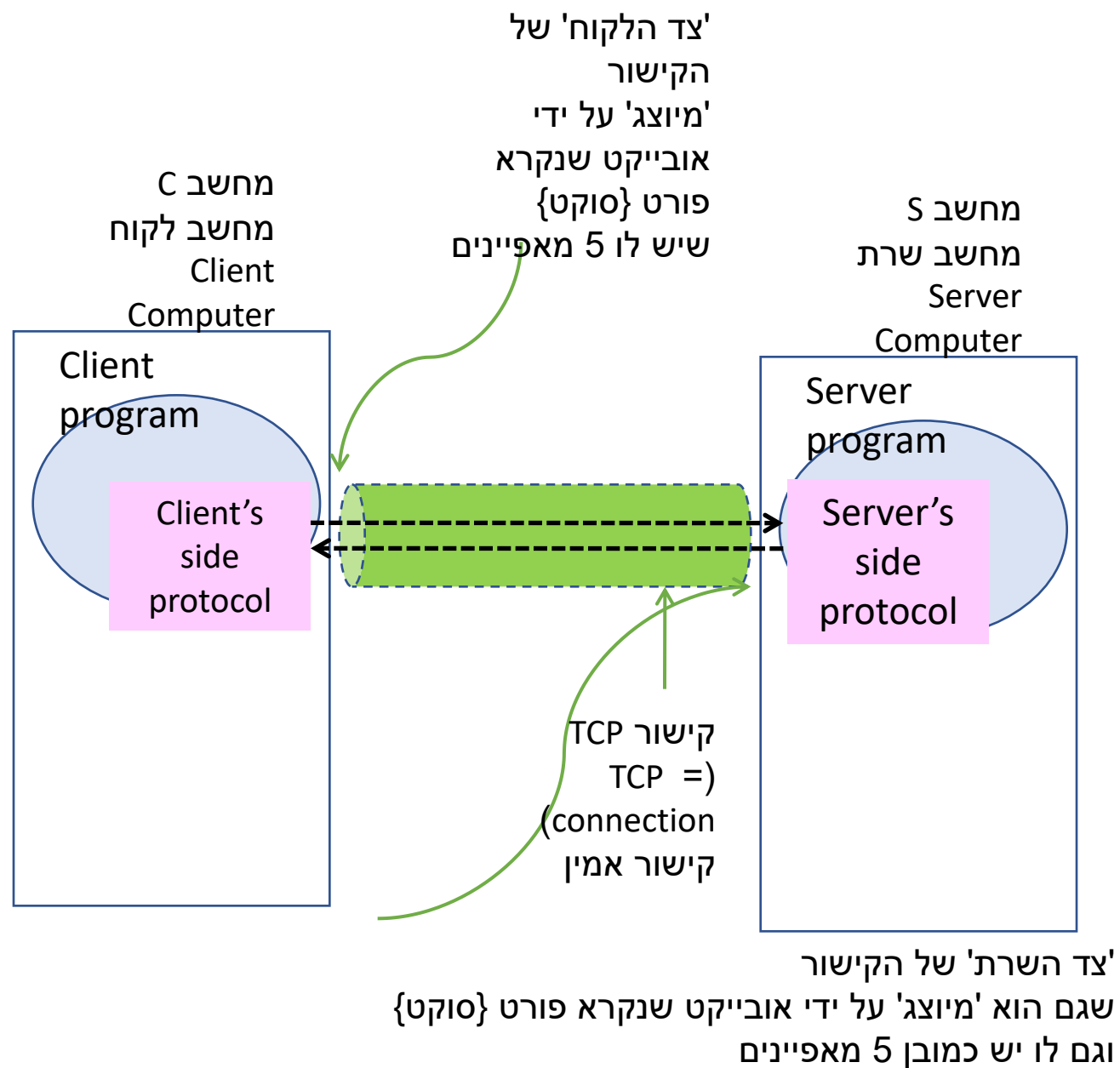
local IP address – כתובת ה- IP של המארח שבו רצה התכנית שמזוהה על ידי ה- *local port*
remote IP address – כתובת ה- IP של שבו 'רצה התכנית השניה' זו שמזוהה על ידי ה- *remote port*

במקרה של פורט שמזהה תכנית לקוח:
ערך המאפיין local IP address הוא כמובן כת' ה- IP של מחשב הלקוח
ערך המאפיין remote IP address הוא כמובן כת' ה- IP של מחשב השרת

ובאופן סימטרי: במקרה של פורט שמזהה תכנית שרת:
ערך המאפיין local IP address הוא כמובן כתובת ה- IP של מחשב השרת
ערך המאפיין remote IP address הוא כמובן כתובת ה- IP של מחשב הלקוח

סיכום:

פורט הוא אובייקט אבסטרקטי שמזהה תכנית שהיא חלק מאפליקציה רשתית ולפורט [= סוקט] יש 5 פרמטרים מאפיינים.



נניח כי L5-prot-client קורא ל- `socket()`
והקריאה חוזרת הצלחה. היינו: הוקצה סוקט לקוח.

הוקצתה רשומת סוקט לקוח
בשלב הזה:

ערכו של השדה `socket type` ברשומה הזו הוא: `TCP`
או, ברוב התכנותי: הערך {הקבוע} `SOCK_STREAM`

ערכו של השדה `local port number` הוא מספר כלשהו בין 0 ל- 65,535
ה- `TCP` במחשב הלקוח, הוא זה ש'אחראי' ליצירת רשומת הסוקט והוא זה ש'אחראי' להקצות ערך אקראי
בשדה `local port number`.

בשדה `local IP address` תשמר כתובת ה- `IP` של מחשב הלקוח.
ערכו של השדה `remote port number` בשלב הזה הוא: `invalid` או: `undefined`
כנ"ל לגבי ערך השדה `remote IP address`.

בשלב הזה, 'לרשות' L5-prot-client מוקצה סוקט לקוח והוא צריך {חייב !!} 'לחבר' אותו [= 'לקשר' אותו] עם סוקט שרת.

איך הוא עושה זאת {ברמת התוכנה}: ע"י קריאה ל- `connect()` לצורך 'הקשר' הזה, L5-prot-client חייב {!!!} לאפיין את 'הקצה השני'. ז"א, חייב לאפיין את: מחשב השרת [= כתובת ה- IP של מחשב השרת] + התכנית 'בצד השני' [= L5-prot-server]

ולכן L5-prot-client אחרי החזרה של `socket()` ולפני הקריאה ל- `connect()`, חייב ל'ספק' את: כת' ה- IP של מחשב השרת מספר הפורט שמייצג את L5-prot-server

ומאיפה לו המידע הזה?
המידע לגבי כת' ה- IP של מחשב הרשת 'מגיעה' ל- L5-prot-client מהמשתמש האנושי.
המשתמש האנושי מקליד host name כקלט לתכנית הלקוח.
ה- host name הוא כמובן של מחשב השרת !!!
וה- L5-prot-client מבקש לתרגם את ה- server host name לכתובת ה- IP של מחשב השרת
ממי מבקש? ממערכת שנקראת DNS [= Domain Name System]
שאחד מתפקידיה הוא לתרגם שמות מחשב לכתובת IP
איך L5-prot-client פונה ל- DNS בבקשת תרגום? יש מספר דרכים. אחת מהן היא על ידי קריאה
לפונקציה gethostname()
הארגומנט שמועבר לה הוא string שמייצג host name
לדוגמא: online.shekar.ac.il או www.goole.com
וערך החזרה של ה- gethostname() הוא:
כשלוש
או: כת' ה- IP המבוקשת.

ש: מאיפה l5-prot-client 'מקבל' את מס' הפורט שמזהה את תכנית השרת {או: l5-prot-server}?
ת: מתואר {חייב להיות מתואר}!!! במסמך האפיון של L5

הערה:

כל מסמך אפיון של פרוט' שכבה 5 חייב לכלול הגדרה של מספר הפורט שמזהה את צד השרת של הפרוטוקול !!
בין אם מדובר בפרוטוקול סטנדרטי {כזה שמסמך האפיון שלו נכתב ע"י גוף התקינה שנקרא {IETF ובין אם מדובר בפרוטוקול לא סטנדרטי.

נזכור שערכו של מספר הפורט שמייצג איזשהו {לא משנה איזה} L5-prot-server הוא בין 0 ל- 65,535 המתכננים של מודל ה- TCP/IP אפיינו והגדירו את כל הנ"ל ובין היתר הם באו וקבעו כי הערכים שבין 0 ל- 1023 'ישמרו לטובת' L5-prot-server סטנדרטיים !! היינו פרוטוקולים שאופיינו על ידי IETF

לקבוצת הערכים האלו ניתן שם ע"י מאפייני מודל ה- TCP/IP
WKPN {או רק WPN}
קיצור של Well Known Port Numbers

http הוא פרוטוקול סטנדרטי.
לכן, מס' הפורט שמייצג את http server הוא שייך לקבוצה הזו של WPN. וספציפית ערכו הוא 80.

https הוא גם פרוטוקול סטנדרטי.
ולכן, גם מס' הפורט שמייצג את https server שייך לקבוצה הזו של WPN.
וספציפית ערכו הוא 431.

דוגמא שלישית:
smtp הוא פרוטוקול סטנדרטי. משמש לשליחת הודעות דוא"ל מ- mail client, כמו לדוגמא תכנית ה-outlook, ל-mail server.
ולכן, גם מס' הפורט שמייצג את smtp server שייך לקבוצה הזו של WPN. וספציפית ערכו הוא 25.

ש: האם מתכנן של פרוטוקול שכבה 5 לא סטנדרטי יכול עקרונית להחליט כי מספר הפורט שמזהה את צד השרת של הפרוטוקול שלו הוא בין 0 ל-1023 למרות כאמור שמדובר בפרוטוקול לא סטנדרטי

תשובה: עקרונית אפשר. לא רצוי.
למה 'לא רצוי'? מכיוון שהתשובה לשאלה האם בפועל מתאפשר להקצות מס' פורט בין 0 ל-1023 לצד השרת של פרוטוקול לא סטנדרטי – התשובה לשאלה הזו תלויה במערכת ההפעלה. או: ב-TCP וב-UDP שממומשים כחלק מהמכלול הכולל של מע' ההפעלה.